**PDF Document Loaders & PyPDFLoader**

## 1. Quick Revision — LangChain Document

Previously, we created a Document manually:

```
from langchain_core.documents import Document

document = Document(
    page_content="Python is a programming language.",
    metadata={
        "source": "python_notes.txt"
    }
)
```

**A Document mainly contains:**

```
Document
   |
   ├── page_content
   |      ↓
   |   Actual Content
   |
   └── metadata
```

↓

**Information About Content**

**Example metadata:**

```
{
   "source": "python_notes.txt"
}
```

## 2. Revision — TextLoader

In the previous session, we used:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader(
   "student_rules.txt",
   encoding="utf-8"
)

documents = loader.load()
```

TextLoader reads:

.txt Files

and converts their contents into:

LangChain Document Objects

Today we move from:

TextLoader

↓

Plain Text Files

to:

PyPDFLoader

↓

PDF Files

## 3. What Is PyPDFLoader?

PyPDFLoader helps LangChain applications read PDF files and represent the extracted content as Document objects.

### Definition

PyPDFLoader is a LangChain Document Loader that reads and parses PDF files using the pypdf library.

Simple flow:

PDF File

↓

PyPDFLoader

↓

Extract Text

↓

LangChain Document Objects

## 4. Why Is a PDF Different from a Normal Text File?

A plain text file mainly contains textual characters.

A PDF is designed to preserve the visual presentation of a document.

A PDF may contain:

- Text
- Multiple pages
- Images
- Tables
- Fonts and formatting
- Page headers and footers
- Different text positions

PDF stands for:

Portable Document Format

PDF text extraction is not always the same as simply reading characters from a .txt file.

TXT File

↓

Mostly Sequential Text

PDF File

↓

Structured Document

↓

Text + Pages + Positions + Formatting

## 5. TextLoader vs PyPDFLoader

| **TextLoader** | **PyPDFLoader** |
|---|---|
| Reads plain text files | Reads PDF files |
| Works with .txt | Works with .pdf |
| Usually one Document per file | Can produce one Document per page |
| Source metadata | Source + PDF-specific metadata |
| Uses TextLoader | Uses PyPDFLoader |

Easy memory:

.txt

↓

TextLoader

.pdf

↓

PyPDFLoader

## 6. Import PyPDFLoader

from langchain_community.document_loaders import PyPDFLoader

## 7. What Is the Role of pypdf?

PyPDFLoader uses another Python library internally:

pypdf

The pypdf library helps process PDF files and extract their text.

Installation:

pip install pypdf

## 8. Creating a PyPDFLoader

Basic syntax:

```
loader = PyPDFLoader(
    file_path="employee_policy.pdf"
)
```

Important parameters include:

| **Parameter** | **Purpose** |
|---|---|
| file_path | Identifies the PDF |
| mode | Controls page-wise or single-document loading |
| pages_delimiter | Separates pages in single mode |
| password | Used for supported encrypted PDFs |

The important point is:

PyPDFLoader

↓

Reads PDF

↓

mode controls how pages

become Document objects

## 9. Understanding mode="page"

mode="page"

In page mode:

Each PDF page is represented as a separate LangChain Document.

Example:

PDF

3 Pages

↓

mode="page"

↓

Document 1 → Page 1

Document 2 → Page 2

Document 3 → Page 3

Normally:

Number of PDF Pages

=

Number of Documents

## 10. PDF Metadata

PDF loaders can provide more metadata than a simple text loader.

Possible information includes:

- Source PDF path
- PDF page index
- Total number of pages
- Page label
- Producer information
- Creator information
- Other available PDF properties

Example:


```
{
   "source": "company_policy.pdf",
   "total_pages": 3,
   "page": 0,
   "page_label": "1"
}
```

## 11. Important Metadata Fields

| Metadata Field | Meaning |
|---|---|
| source | PDF source location |
| page | Page index starting from 0 |
| page_label | Page label defined by the PDF |
| total_pages | Total number of pages |
| producer | PDF producer information, when available |
| creator | PDF creator information, when available |

## 12. Very Important — page Starts from Zero

Suppose a PDF has three pages.

Actual Page  Metadata page

---  ---

Page 1  0

Page 2  1

Page 3  2

So:

document.metadata["page"]

is a zero-based page index.

If we want a normal human-readable page number:

```
page_number = (
    document.metadata["page"] + 1
)
```

**13. page vs page_label vs total_pages**

page

↓

Zero-based page index

page_label

↓

Label defined by PDF

total_pages

↓

Actual total page count

Example:

page = 0

page_label = "1"

total_pages = 3

Main conclusion:

PDF metadata helps us trace extracted text back to the correct location inside the original PDF.

## 14. Understanding mode="single"

Example:

```
loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="single"
)
```

In single mode:

Text extracted from all PDF pages is combined into one Document object.

Example:

PDF

Page 1

Page 2

Page 3

↓

mode="single"

↓

One Document

↓

Combined Page Text

## 15. mode="page" vs mode="single"

### Page Mode

Page 1 → Document 1

Page 2 → Document 2

Page 3 → Document 3

### Single Mode

Page 1

Page 2

Page 3

↓

One Combined Document

Easy memory:

page

↓

Many Documents

single

↓

One Document

## 16. Understanding pages_delimiter

Suppose we use:

mode="single"

The PDF has three pages.

All page text will be combined.

But we may want to clearly identify page boundaries.

For this we can use:

pages_delimiter

Example:

```
loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="single",
    pages_delimiter="\n--- PAGE BREAK ---\n"
)
```

The resulting text may look like:

Page 1 information

--- PAGE BREAK ---

Page 2 information

--- PAGE BREAK ---

Page 3 information

pages_delimiter helps preserve visible separation between extracted page texts in single mode.

## 17. Important PyPDFLoader Syntax

| **Requirement** | **Syntax** |
|---|---|
| Import loader | from langchain_community.document_loaders import PyPDFLoader |
| Create loader | PyPDFLoader(file_path="sample.pdf") |
| Page-wise loading | mode="page" |
| Combined loading | mode="single" |
| Load all Documents | loader.load() |

| | |
|---|---|
| Iterate lazily | loader.lazy_load() |
| Read page text | document.page_content |
| Read page index | document.metadata["page"] |
| Read page count | document.metadata["total_pages"] |
| Read page label | document.metadata.get("page_label") |
| Read source | document.metadata["source"] |
| Page separator | pages_delimiter="..." |

## 18. Five Important Observations

1. PyPDFLoader is used for PDF documents.
2. Page mode is the default in the API described in these notes.
3. One PDF may produce multiple Document objects.
4. PDF page metadata uses zero-based indexing.
5. Scanned PDFs may require OCR to obtain useful text.

## 19. Text-Based PDF vs Scanned PDF

There are two important types of PDFs.

| **Text-Based PDF** | **Scanned PDF** |
|---|---|
| Contains extractable text | May contain only images |
| Text can often be selected | Text may not be selectable |
| PyPDFLoader can commonly extract text | Basic extraction may return little/no text |
| OCR usually not required | OCR may be required |

## 20. What Is OCR?

OCR stands for:

Optical Character Recognition

OCR helps convert text visible inside images into machine-readable text.

Simple flow:

Scanned Page Image

↓

OCR

↓

Recognized Text

According to the source notes, default PyPDFLoader text extraction does not automatically perform OCR on scanned pages.

## 21. Program 1 — Our First PyPDFLoader Program

### Objective

Load a single-page PDF and access:

- Content
- Source
- Page index
- Total pages
- Metadata

```
from pathlib import Path


from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# SPECIFY PDF FILE PATH
```

```
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "library_notice.pdf"
)


# ========================================
# CREATE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path=str(pdf_path)
)


# ========================================
# LOAD PDF DOCUMENTS
# ========================================
```

```
documents = loader.load()



# ========================================
# DISPLAY RETURN INFORMATION
# ========================================

print("RETURN TYPE")
print("-----------")

print(
   type(documents).__name__
)


print("\nTOTAL DOCUMENTS")
print("---------------")

print(
```

```
    len(documents)
)



# ========================================
# ACCESS FIRST DOCUMENT
# ========================================

document = documents[0]



# ========================================
# DISPLAY EXTRACTED CONTENT
# ========================================

print("\nPAGE CONTENT")
print("------------")

print(
    document.page_content
```

```
)


# ========================================
# DISPLAY METADATA
# ========================================

print("\nMETADATA")
print("--------")

print(
   document.metadata
)


# ========================================
# DISPLAY SOURCE INFORMATION
# ========================================

print("\nSOURCE FILE")
```

```
print("-----------")

print(
    Path(
        document.metadata["source"]
    ).name
)



# ========================================
# DISPLAY PDF PAGE INFORMATION
# ========================================

print("\nPAGE INFORMATION")
print("----------------")

print(
    "Page Index:",
    document.metadata["page"]
)
```

```
print(
    "Total Pages:",
    document.metadata["total_pages"]
)
```

**22. Program 1 Flow**

library_notice.pdf

↓

PyPDFLoader

↓

loader.load()

↓

List of Documents

↓

documents[0]

↓

page_content

+

metadata

## 23. Important Observation

documents = loader.load()

returns:

List

Then:

document = documents[0]

returns:

Document

So:

documents

↓

List

documents[0]

↓

Document

## 24. Program 2 — Loading a Multipage PDF

### Objective

Load a three-page PDF and access each page separately.

```
from pathlib import Path

from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# SPECIFY PDF FILE
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)
```

```
# ========================================
# CREATE PAGE-WISE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path=str(pdf_path),
    mode="page"
)



# ========================================
# LOAD PDF PAGES
# ========================================

documents = loader.load()



# ========================================
# DISPLAY TOTAL DOCUMENTS
# ========================================
```

```
print("COMPANY POLICY PDF")
print("==================")


print(
    "Total Documents:",
    len(documents)
)


# ========================================
# DISPLAY EACH PAGE
# ========================================

for document in documents:

    page_index = (
        document.metadata["page"]
    )
```

```
page_number = (
    page_index + 1
)


text = " ".join(
    document.page_content.split()
)


print(
    "\nPAGE",
    page_number
)

print(
    "--------------------"
)
```

```
    print(
        "CONTENT:",
        text
    )
```

**25. Understanding This Line**

```
text = " ".join(
    document.page_content.split()
)
```

Let us understand it step by step.

First:

```
document.page_content.split()
```

breaks the text into words using whitespace.

Example:

```
"Python    is


easy"
```

becomes approximately:

```
[
    "Python",
```

"is",

"easy"

]

Then:

" ".join(...)

joins the words using one space.

Final result:

Python is easy

So this line helps produce cleaner one-line text for display.

**26. Why Add 1 to Page Index?**

page_index = document.metadata["page"]

page_number = page_index + 1

Because:

Metadata Page Index

↓

Starts at 0

while students normally expect:

Displayed Page Number

↓

Starts at 1

Example:

page index 0 → page number 1

page index 1 → page number 2

page index 2 → page number 3

## 27. Program 3 — Understanding PDF Metadata in Detail

### Objective

Examine:

- Source
- Page index
- Display page
- Page label
- Total pages

```
from pathlib import Path

from langchain_community.document_loaders import (
    PyPDFLoader
)
```

```
# ========================================
# SPECIFY PDF FILE
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)


# ========================================
# CREATE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path=str(pdf_path),
    mode="page"
)
```

```
# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY PDF SUMMARY
# ========================================

print("PDF METADATA REPORT")
print("====================")



print(
    "Total Documents:",
    len(documents)
```

```
)


# ========================================
# DISPLAY METADATA FOR EACH PAGE
# ========================================

for document in documents:

    metadata = document.metadata


    print("\nPAGE METADATA")
    print("-------------")


    print(
        "Source:",
        Path(
            metadata["source"]
```

```
    ).name
)


print(
    "Page Index:",
    metadata["page"]
)


print(
    "Display Page:",
    metadata["page"] + 1
)


print(
    "Page Label:",
    metadata.get(
        "page_label",
```

```
            "Not available"
        )
    )


    print(
        "Total Pages:",
        metadata["total_pages"]
    )
```

**28. Possible Output**

```
PDF METADATA REPORT
==================
Total Documents: 3

PAGE METADATA
-------------
Source: company_policy.pdf
Page Index: 0
Display Page: 1
```

Page Label: 1

Total Pages: 3

PAGE METADATA

---

Source: company_policy.pdf

Page Index: 1

Display Page: 2

Page Label: 2

Total Pages: 3

PAGE METADATA

---

Source: company_policy.pdf

Page Index: 2

Display Page: 3

Page Label: 3

Total Pages: 3

## 29. Why Is Metadata Important?

Suppose RAG returns some information such as:

Employees receive 20 days leave.

Metadata can help us identify:

Which PDF?

↓

company_policy.pdf

Which Page?

↓

Page 2

Therefore:

Extracted Content

+

Metadata

↓

Traceability

The source notes emphasize that PDF metadata preserves useful source and page information.

**30. Program 4 — mode="page" vs mode="single"**

```
from pathlib import Path

from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# SPECIFY PDF FILE
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)



# ========================================
# PAGE MODE
```

```
# ========================================

page_loader = PyPDFLoader(
    file_path=str(pdf_path),
    mode="page"
)


page_documents = (
    page_loader.load()
)


# ========================================
# SINGLE MODE
# ========================================

single_loader = PyPDFLoader(
    file_path=str(pdf_path),
    mode="single",
```

```
    pages_delimiter=
    "\n--- PAGE BREAK ---\n"
)



single_documents = (
    single_loader.load()
)



# ========================================
# COMPARE DOCUMENT COUNTS
# ========================================

print(
    "PAGE MODE DOCUMENTS"
)

print(
    "-------------------"
```

```
)

print(
    len(page_documents)
)


print(
    "\nSINGLE MODE DOCUMENTS"
)

print(
    "--------------------"
)

print(
    len(single_documents)
)
```

```
# ========================================
# ACCESS COMBINED DOCUMENT
# ========================================

document = (
    single_documents[0]
)



# ========================================
# CHECK PAGE METADATA
# ========================================

print(
    "\nSINGLE MODE METADATA"
)

print(
    "--------------------"
)
```

```
print(
    "Total Pages:",
    document.metadata["total_pages"]
)


print(
    "Page Index Available:",
    "page" in document.metadata
)


# ========================================
# DISPLAY COMBINED PAGE CONTENT
# ========================================

print(
    "\nCOMBINED PDF CONTENT"
```

```
)

print(
    "--------------------"
)


pages = (
    document.page_content.split(
        "--- PAGE BREAK ---"
    )
)


for number, text in enumerate(
    pages,
    start=1
):

    clean_text = " ".join(
```

```
        text.split()
    )



    print(
        "\nSECTION",
        number
    )

    print(
        clean_text
    )
```

## 31. Program 4 Main Flow

```
company_policy.pdf
        |
        ├──────────────────────────┐
        |                          |
        ↓                          ↓
   mode="page"              mode="single"
```

↓ ↓

Document per Page One Combined Document

↓ ↓

Page Metadata pages_delimiter

Available Separates Text

## 32. Document Count Comparison

Suppose a PDF has three pages.

**Page Mode**

3 Pages

↓

3 Documents

**Single Mode**

3 Pages

↓

1 Document

Easy memory:

page

↓

Page → Document

single

↓

PDF → Document

**33. Why Use pages_delimiter in Single Mode?**

Without a delimiter:

Page 1 text Page 2 text Page 3 text

may be harder to visually separate.

With:

pages_delimiter="\n--- PAGE BREAK ---\n"

we can get:

Page 1 text

--- PAGE BREAK ---

Page 2 text

--- PAGE BREAK ---

Page 3 text

## 34. Program 5 — Lazy Loading and Empty Text Detection

### Objective

Process PDF pages one by one and identify pages that contain no extractable text.

```
from pathlib import Path


from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# SPECIFY PDF FILE
# ========================================


pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)
```

```
# ========================================
# CREATE PAGE-WISE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path=str(pdf_path),
    mode="page"
)


# ========================================
# PROCESS PDF PAGES LAZILY
# ========================================

processed_pages = 0


for document in loader.lazy_load():
```

```
processed_pages += 1


page_number = (
    document.metadata["page"] + 1
)


text = " ".join(
    document.page_content.split()
)


print(
    "\nPAGE",
    page_number
)

print(
    "--------------------"
```

```
)


if text:

    print(
        "STATUS: Text extracted"
    )

    print(
        "CONTENT:",
        text
    )


else:

    print(
        "STATUS: No extractable text"
    )
```

```
        print(
            "Check whether the page is blank,"
            " scanned, or requires a different"
            " extraction approach."
        )


# ========================================
# DISPLAY TOTAL
# ========================================

print(
    "\nTOTAL PAGES PROCESSED"
)

print(
    "--------------------"
)
```

```
print(
    processed_pages
)
```

**35. What Does lazy_load() Do?**

loader.lazy_load()

allows us to process Document objects as they are yielded.

Concept:

PDF

↓

lazy_load()

↓

Page Document 1

↓

Process

↓

Page Document 2

↓

Process

↓

Page Document 3

↓

Process

This is different from:

loader.load()

which collects the Documents into a list.

## 36. load() vs lazy_load()

| **load()** | **lazy_load()** |
|---|---|
| Returns collected Documents | Yields Documents |
| Easy for normal loading | Useful for iterative processing |
| Often work with a list | Often work with a loop |

Simple:

load()

↓

Collect First

lazy_load()

↓

Process as Yielded

## 37. Why Detect Empty Text?

Suppose:

```
text = " ".join(
    document.page_content.split()
)
```

returns:

""

Then that page contains no extracted text.

Possible reasons from the notes include:

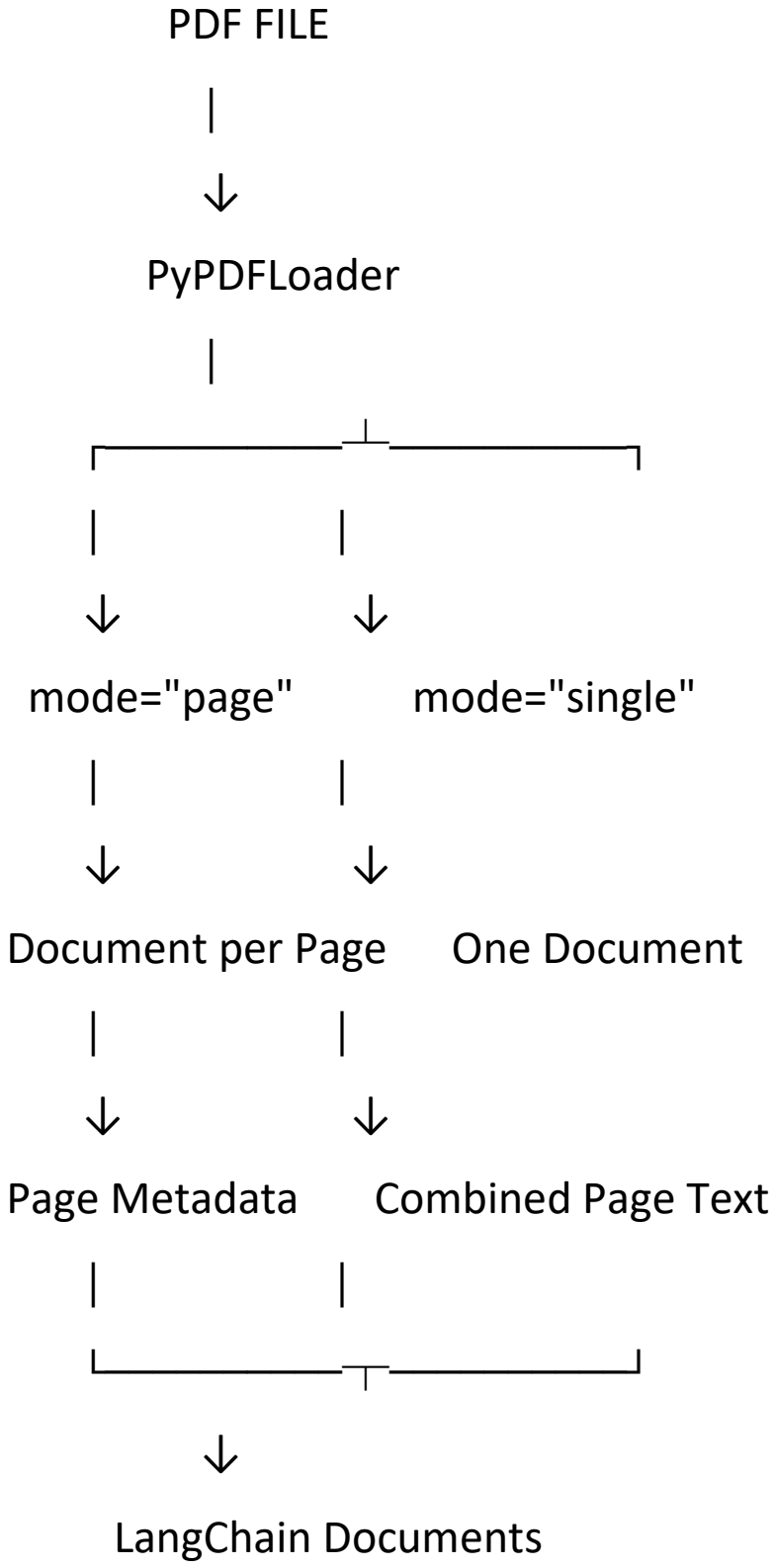
Blank Page

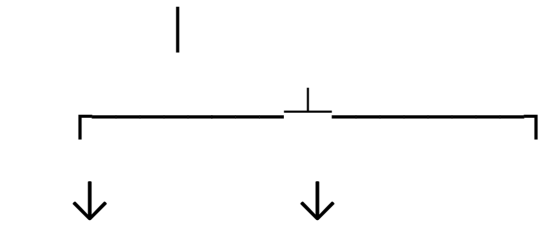
Scanned Page

Different Extraction

Approach Required

## 38. Complete PyPDFLoader Architecture






## 39. Important Notes

1. PyPDFLoader is a LangChain Document Loader for PDF files.
2. It uses pypdf for PDF processing.
3. It returns LangChain Document objects through the common loader interface.
4. The source notes describe the current default as mode="page".
5. Page mode normally creates one Document per PDF page.
6. PDF metadata includes information such as source, page index and total page count.
7. page metadata starts at zero.
8. mode="single" combines extracted page text into one Document.
9. .lazy_load() supports processing page Documents as they are yielded.

10. Scanned PDFs and complex layouts may require specialized extraction or OCR.

## 40. TextLoader vs PyPDFLoader — Final Comparison

| **Concept** | **TextLoader** | **PyPDFLoader** |
|---|---|---|
| File type | .txt | .pdf |
| Main purpose | Read plain text | Extract PDF text |
| Output | Document objects | Document objects |
| Page concept | Usually not applicable | Important |
| PDF metadata | No | Yes |
| Page mode | No | Yes |
| Single mode | No | Yes |
| OCR | Not relevant to plain text | May be needed for scanned PDFs |

## 41. Common Mistake — Thinking One PDF Always Means One Document

Wrong:

1 PDF

=

1 Document

Not always.

In:

mode="page"

a three-page PDF normally produces:

3 Documents

## 42. Common Mistake — Thinking page=1 Means First Page

Remember:

page = 0

↓

First Page

page = 1

↓

Second Page

because page metadata uses zero-based indexing.

## 43. Common Mistake — Confusing Page Index with Total Pages

document.metadata["page"]

means:

Current Page Index

whereas:

document.metadata["total_pages"]

means:

Total PDF Page Count

## 44. Common Mistake — Thinking Single Mode Gives Multiple Documents

Wrong.

mode="single"

means:

All Extracted Page Text

↓

One Document

## 45. Common Mistake — Thinking pages_delimiter Splits into Documents

pages_delimiter does not mean:

Create Multiple Documents

It means:

Put a Visible Separator

Between Combined Page Text

when using single mode.

## 46. Common Mistake — Thinking PyPDFLoader Automatically Solves Every Scanned PDF

A scanned PDF may mainly contain page images.

Therefore basic text extraction may return:

Very Little Text

or:

No Extractable Text

Such cases may require OCR or another extraction approach.

## 47. Student Memory Formula

PDF

↓

PyPDFLoader

↓

Document Objects

↓

page_content

+

metadata

For page mode:

PDF

↓

Pages

↓

One Document per Page

For single mode:

PDF

↓

All Pages

↓

One Combined Document

## 48. MCQ 1

A PDF contains five pages.

PyPDFLoader uses:

mode="page"

How many Document objects will it normally produce?

A. 1
B. 5
C. 10
D. 0

**Answer: B — 5**

**Explanation**

Page mode normally creates one Document for each PDF page.

**Learning Conclusion**

**One page, one Document — that is the simple rule of page mode.**

## 49. MCQ 2

What does:

document.metadata["page"]

represent?

A. Total number of PDF pages
B. Source filename
C. Zero-based PDF page index
D. PDF file size

**Answer: C — Zero-based PDF page index**

**Explanation**

The first page has index 0, the second page has index 1, and so on.

**Learning Conclusion**

**Computers may start counting at zero; users normally start counting at one.**

## 50. MCQ 3

What happens when PyPDFLoader uses:

mode="single"

on a multipage PDF?

A. It deletes extra pages.
B. It creates one Document containing combined extracted

text.

C. It creates one Document for every line.

D. It performs OCR automatically.

**Answer: B**

**Explanation**

Single mode combines extracted page text into one Document.

**Learning Conclusion**

**Page mode separates; single mode combines.**

## 51. Five Final Revision Points

1. PyPDFLoader

↓

Loads PDF files

2. mode="page"

↓

One Document per Page

3. mode="single"

↓

One Combined Document

4. metadata["page"]

↓

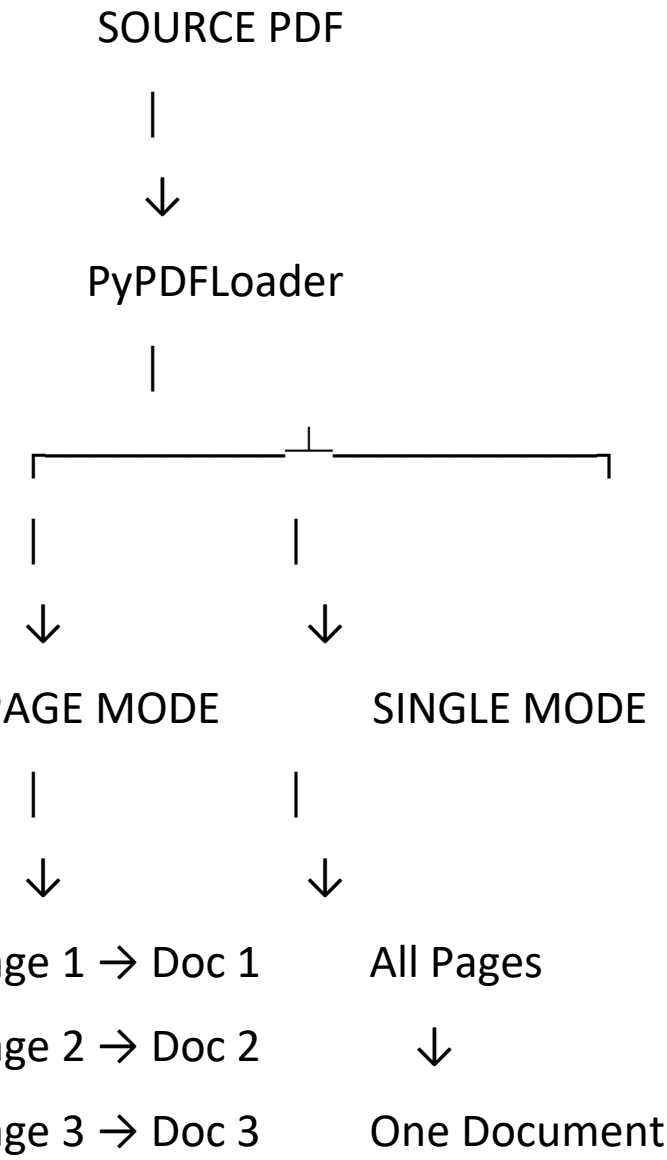
Zero-Based Page Index

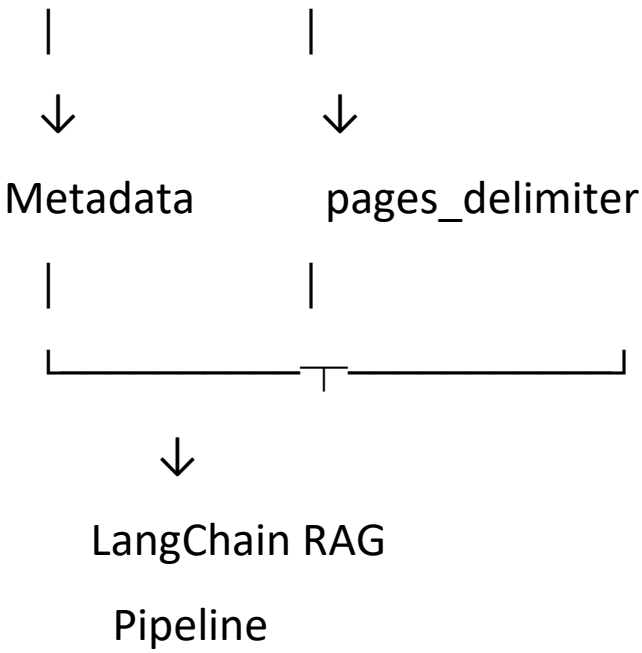
5. Scanned PDF

↓

May Require OCR

## 52. Final Master Diagram






## 53. One-Line Conclusion

**PyPDFLoader reads PDF files and converts their extracted content into LangChain Document objects; page mode keeps pages separate, while single mode combines the page text into one Document.**